

Can Multi-LLM Reasoning Improve Decision-Making?

Evidence from Elastic GPU Cluster Scheduling

Lay Kim Seng

Cyberscience Center, Tohoku University

lay.kim.seng.q4@dc.tohoku.ac.jp

Abstract—A GPU job that can be resized makes two demands on a scheduler: which job to start, and how many devices to give it. We ask whether reasoning structure helps with the second, on the MIT Supercloud accounting log replayed through a malleability-aware simulator on an 80-GPU pool. Sizing is where the leverage is: across twelve held-out windows the four queue-ordering rules we test span under one percentage point of deadline violations while sizing spans 15.1 to 47.1, and the sizing rule attaining the highest occupancy we measure, 0.988 against 0.890, is simultaneously the worst on every service axis, because sublinear speed-up converts capacity into busy-ness rather than completed work. Counterfactual enumeration of every policy from sampled states bounds what any state-dependent selector could add at 2.03 points for queue ordering and 0.07 for auctions, with a median of zero. Asked to select policies from that same observable state, an 8B model carries 8.70 points of mean regret against the deterministic policy’s 1.05 and emits an illegal action on 11 of 24 held-out states; low-rank fine-tuning changes nothing. A pre-registered 2×2 test gives one narrower qualification: role separation is 0.38 deadline points worse without the market but 1.42 points better with it, a -1.79 -point interaction that is directionally consistent (exact sign-flip Holm $p = 0.0469$) though its paired- t interval $[-3.84, 0.25]$ crosses zero and the structure main effect is null. Role separation may thus be complementary with a market, but it does not win in general; numeric rationing still belongs by default to validated deterministic code.

Index Terms—GPU scheduling, large language models, resource allocation, multi-agent systems, cluster traces

I. INTRODUCTION

AI training and inference are the dominant consumers of GPU clusters, and they are *elastic*: a training job can run on more or fewer devices, trading device-hours against completion time. A scheduler on such a workload therefore makes two decisions per job, not one — *which* job to start, and *how large* to make it.

A trace records that a job used g devices for t seconds; it does not record what that job would have done on $2g$ or $g/2$, so any study of the sizing decision must supply that counterfactual. We supply it with a speed-up model anchored on each job’s observed point, and sweep its parallel fraction rather than fixing it, so that the rigid replay is the same model evaluated at the size each job really used.

Our contributions, all on the MIT Supercloud GPU trace [2] replayed through a malleability-aware batch simulator [7], are:

- 1) **Occupancy is an anti-metric for elastic workloads** (§IV-B). Sizing moves service quality by an order of

magnitude more than ordering does, and the highest-occupancy rule is the worst-service rule in every window we measure. We recommend reporting device-hours per unit of completed work instead.

- 2) **Language-model policy selection does not match deterministic code** (§IV-D, §IV-E). On held-out windows the ordering rule is worth under one point while sizing is worth a factor of three and the auction’s pricing rule six points, and the advantage of auction pricing over ordering is a single-window tail effect. Counterfactual enumeration bounds the available headroom at 2.03 and 0.07 points with a median of zero, and neither a zero-shot model nor a low-rank fine-tune comes near it: eight times the deterministic policy’s regret, and an illegal action on 11 of 24 held-out states.
- 3) **Role separation is complementary with the market, not beneficial in general** (§IV-C). On twelve untouched test windows it is 0.38 deadline points worse without the market but 1.42 points better inside it. The pre-registered interaction is -1.79 points (exact Holm $p = 0.0469$), while its paired- t interval crosses zero and the structure main effect is null.
- 4) **A protocol that makes such a comparison auditable** (§IV-A). The selector study is frozen before execution: workload-only window selection, a hashed manifest of trace, worlds, code and configuration, a fail-closed gate, and amendments recorded rather than absorbed. A post-core bounded-debate pilot (§IV-F) further shows that the worth of auditable disagreement depends on contention: it is inert on a slack window and 9.8 deadline points better than a single selector on a saturated one, on training windows at one seed.

A qualified result of this kind is only worth reporting if the comparison was fair, so §IV-A sets out the protocol: windows chosen from workload statistics alone, a deterministic baseline selected on validation and never on the reserved split, an identical action space and model configuration across structures, and a budget-matched control separating role separation from the effect of spending three times the calls.

II. RELATED WORK

LLM-driven scheduling. LLMsched [1] repairs a single LLM’s proposal with an ILP and reports large gains over

Kubernetes. We inherit the reason-then-guarantee spine but invert its default: on numeric rationing the deterministic layer should be the proposer, not just the repairer, and the LLM should be gated behind a contextual trigger.

Learning-based schedulers. DRL schedulers (DeepRM [3], Decima [4]) learn allocation policies but are black-box and brittle under distribution shift. On our interface a tabular Q-learner’s episode-return variance dominates the action effect at realisable sample counts.

Multi-agent LLM structures. Multiagent debate [8] and self-consistency [9] are usually evaluated on QA benchmarks, and LLM-as-judge protocols [10] inherit that setting; budget-matched controls are rare, so a reported gain is rarely separable from the extra calls that buy it. Every structure comparison we report carries such a control.

GPU cluster traces. Public traces record allocations, not counterfactual sizes: neither Supercloud [2] nor the other widely used GPU trace [6] states what a job would have done at a different size, which is why an elastic study must supply a speed-up model.

Malleability and mechanism design. Runtime elasticity is rarely deployed in production [5]; our partial-allocation model rests on it and we state that dependence. Uniform-price auctions are the classical route to truthful revelation, and automated negotiation for resource allocation is a classical subject [11], with prediction-guided *selective* negotiation explored in one-shot fair division [12]; our contribution is measurement in a closed loop containing declarative LLM agents.

III. CONTROLLED POLICY-SELECTOR ARCHITECTURE

Both reasoning structures read the same state and emit the same action. At each 1,800-second selection point the model sees available capacity and scheduler-visible attributes of pending and running jobs: arrival and waiting time, requested walltime and GPU count, priority, deadline pressure, legal malleable bounds, and scaling indicators. It never sees true runtime or future arrivals.

Single Referee. One model returns strict JSON containing one global ordering policy, a default sizing rule, and optional job-specific sizing overrides. The parser expands this into exactly one action for every affected job.

Demand–Supply–Referee. Demand identifies urgent, deadline-pressured, scalable, and shrink-tolerant jobs. Supply describes scarcity, queue pressure, resize cost, and fairness. These agents cannot allocate GPUs; their analyses and the original state are passed to a Referee whose output schema is identical to the Single arm.

Bounded policy debate (post-core, exploratory). After the frozen factorial completed, we implemented a stricter one-round protocol to address consensus theatre and unconstrained judging. Both advocates share the dimensions (deadline protection, wait reduction, capacity efficiency, fairness), but receive disclosed weights $\mathbf{w}_D = (.45, .35, .10, .10)$ and $\mathbf{w}_S = (.05, .10, .50, .35)$, whose L_1 distance is 1.3. Their red lines are machine-readable and evaluated from simulator

state: when at least one waiting job has negative laxity, Demand must retain deadline pricing; when queue pressure reaches 0.75, Supply may not choose maximum-fit sizing. Openings are independent. If they differ, each advocate may retain, copy the other opening, or change exactly one of ordering and sizing. An advocate states only the policy it now stands behind; code compares it against both openings, derives which of the three moves it is, and refuses a policy that is neither opening and differs from the advocate’s own in both components. Code ratifies an opening or post-rebuttal consensus directly; otherwise the Referee must copy one surviving global proposal and cannot add job-level overrides. Invalid moves retain the last valid opening, while an invalid final ruling invokes the same deterministic floor. The resulting package records weights, red-line state, openings, concessions, rejected moves, candidate policies, dissent, resolution source and fallback use. Thus a round costs two calls on opening consensus, four when rebuttals converge, and five only when unresolved disagreement reaches the Referee.

The action set. Both structures choose from the same two-part action, and the two policy families are matched in size so that neither referee simply has more options than the other.

The *queue-ordering* family decides which waiting job is considered first: first-come first-served, least-laxity-first on the requested walltime, ageing fair-share on accumulated per-user waiting, and priority-tier ordering that clears the high-priority tier first. The *auction* family replaces that comparison with a single uniform-price clearing and varies only what a job’s claim is priced on: the time it has already waited, its deadline pressure, its owner’s waiting-service deficit, or its priority tier. Holding the clearing rule fixed isolates the valuation, which is the design decision an operator actually makes.

Crossed with either family, four sizing rules decide how many devices a started job receives, and govern later resize points as well. *Requested size* gives a job exactly what its owner asked for and makes it wait until that much is free; this reproduces the rigid replay and is the control. *Queue-proportional* sizing shares the free pool across the waiting queue, so a job may start smaller when others are waiting and grow when they are not. *Maximum-fit* sizing gives every job the largest legal size that fits at the moment it starts. *Benefit-gated* sizing changes a running job’s size only when the estimated gain, measured in seconds against the longest current wait and charged for the extra runtime that shrinking causes, exceeds a fixed threshold of 300 s; its estimates use requested walltimes only, never true runtimes.

A shared validator rejects malformed actions, enforces the 80-GPU capacity and each job’s legal bounds, and invokes a counted deterministic fallback, so the comparison changes only the policy family and the reasoning structure.

IV. EVALUATION

Setup. We replay the MIT Supercloud GPU trace [2] (a 235-day Slurm accounting log) through ElastiSim [7], a batch-system simulator for malleable workloads. One simulated node

is one GPU, so a job that used g GPUs becomes a g -node job; its work is set so that it reproduces its recorded runtime exactly. The scheduler sees each job’s arrival, size and *requested* walltime, never its true runtime — the same information a production batch system has. Capacity is a fixed 80-GPU pool, so a window’s offered load is a property we *measure* rather than a consequence of sizing the cluster to fit it.

Window selection is pre-registered. Every 24 h window is described by workload statistics alone — job count, GPU-hours, offered load, high-priority fraction, runtime and size distribution — with no scheduler executed. Of 5,613 hourly starts, 4,312 carry at least 100 jobs; these are stratified into four offered-load bands crossed with three high-priority bands, and windows are drawn per cell under a fixed seed subject to a non-overlap constraint. The 12 so drawn span days 41–222, offered loads $0.75\text{--}4.97\times$ and priority fractions $0.7\text{--}58\%$, and carry the deterministic control study of §IV-D; the selector factorial of §IV-A draws its own 48 by the same procedure. Each is preceded by 12 h of warm-up arrivals, which the arm under test also schedules but which are excluded from scoring, so no run begins on an empty cluster. Selection never sees a scheduler result.

Elasticity model. A job may be *moldable*: its size is chosen once, at launch, and fixed thereafter. Speed-up follows Amdahl’s law anchored on the observed point — a job seen at g GPUs reproduces its recorded runtime exactly at g , so the rigid replay is this same model evaluated at the size the job really ran, and the parallel fraction s is swept rather than assumed ($s=1$ is rigid at every size, $s=0$ is linear speed-up). Results below use $s=0.7$, a deliberately conservative value: scaling is sublinear, so growing a job costs more device-time than it saves. We report SLA- k (turnaround exceeding $k\times$ true runtime) at $k \in \{2, 5, 10\}$, mean waiting time, device-hours actually spent, walltime kills, and occupancy over the measured window.

A. Frozen factorial protocol

The core experiment uses 48 non-overlapping 24-hour windows, each with 12 hours of warm-up, selected from workload characteristics before scheduling outcomes are inspected. They are frozen as 24 train, 12 validation, and 12 test windows. Every scored window contains at least 100 jobs. Jobs are malleable at ten resize points with bounds $1 \leq g_i(t) \leq \min(80, 4g_i^{\text{trace}})$, $s = 0.7$, and no modeled resize overhead. Synthetic deadlines are generated once as $D_i = a_i + W_i^{\text{req}}$; a missing walltime receives the fixed 86,400-second estimate. Deadline violation rate is the primary outcome.

The four inferential conditions cross the two reasoning structures with the two policy families. Qwen2.5-14B is held fixed at temperature 0.1 and a 400-token output limit, and the two structures see an identical state, action set, parser, and validator, so the only difference between them is that the role-separated arm spends three calls on two analyses and a ruling where the single referee spends one. Thirty-two deterministic controls exhaustively cross the four ordering

or pricing rules with the four sizing rules in each family. Validation selects the best fixed policy per family; the reserved split is opened once. The inferential unit is the window. The preregistered analysis contains exactly three two-sided paired contrasts on the reserved split: the family main effect, the structure main effect, and their difference-in-differences interaction, with paired- t 95% confidence intervals, exact sign-flip tests, and Holm correction across those three.

Role separation spends three calls where the single referee spends one, so the design also carries a *budget-matched control*: the single prompt sampled three times with a majority vote. It matches call count and output budget but necessarily not temperature, 0.8 against 0.1, since a vote over three samples at 0.1 returns near-identical answers and collapses into the single-call arm. It was added after a pilot on the four highest-load *training* windows, run to decide whether the remaining inference was worth its cost; training windows were used so that no pilot observation touches the reserved split, and no pilot number is reported here. Two further amendments are recorded with the protocol rather than absorbed: a correction to the feasibility assertion, which had excluded completed but not killed jobs from ownership accounting and aborted valid runs without affecting any policy or metric, and a reduction from three inference repetitions to one, since at 1,703 s per role-separated window against 251 s for the single referee the three-repetition design needed a further 64 hours. That reduction merges decoding noise into the window-level differences, which makes the paired tests more conservative rather than less.

Every result row stores hashes of the trace, the generated world, the implementation, the manifest and the revision, and a fail-closed gate re-checks all of them, with the split, window non-overlap, per-job deadlines, malleable bounds and speed-up anchoring, before any run executes. Superseded hashes are retained, so rows produced before an amendment stay verifiable instead of being restamped. Runtime assertions enforce capacity, malleable bounds, non-negative remaining work and single completion at every scheduler invocation.

B. Sizing dominates ordering, and occupancy inverts

Before handing the sizing decision to a selector, we establish what it is worth, and what the obvious objective for it costs. Holding ordering at first-fit and varying only sizing over the same 12 windows, requested-size sizing spends 3,653 device-hours at 0.890 occupancy for 22.3% slowdown violations; queue-proportional spends 3,339 at 0.964 for 32.5%; maximum-fit spends 9,581 at 0.988 for 76.2%. Paired against requested size, maximum-fit costs $+5,929 \pm 1,707$ device-hours, $+232,451 \pm 60,823$ s of waiting and $+53.9 \pm 12.3$ points of violations, an order of magnitude beyond anything ordering moved. The rule with the *highest* occupancy is thus the worst on every service axis, in 12 of 12 windows, because under sublinear speed-up a job given $4\times$ the devices at $s=0.7$ finishes about $1.6\times$ sooner while consuming $4\times$ the capacity. Occupancy records the resulting busy-ness and is blind to the waste: on one low-load window it reads 1.000 for all

TABLE I
COMPLETED FROZEN 2×2 ON 12 HELD-OUT WINDOWS. ENTRIES ARE
MEAN SYNTHETIC DEADLINE VIOLATIONS; DIFFERENCES ARE
PERCENTAGE POINTS AND LOWER IS BETTER.

family	Single (%)	Multi (%)	Multi–Single
queue ordering	14.750	15.125	+0.375
auction pricing	9.625	8.208	−1.417

three sizing rules while their mean waiting spans 21,429 to 244,781 s. We therefore report device-hours per unit of completed work alongside waiting time, and treat occupancy as diagnostic rather than as an objective.

The obvious fix for over-sizing does not work either. Queue-proportional sizing does not beat requested size on waiting ($+1,684 \pm 10,847$ s, $p=0.77$) or device-hours (-313 ± 637 , $p=0.36$) and is significantly worse on violations ($+10.2 \pm 8.3$ points, $p=0.035$), because shrinking a job lengthens it and can push it past its requested limit. What remains open is sizing under genuine runtime uncertainty, on a trace whose only estimate is uninformative, and that is the decision we hand to the selectors below.

C. Selector conditions: what the completed runs show

All 48 confirmatory cells completed: twelve untouched test windows crossed with Single or Demand–Supply–Referee reasoning and with the queue-ordering or auction family. Table I gives the window-level means for the primary outcome.

The three pre-registered paired contrasts are not three versions of the same conclusion. Averaging over structures, the market main effect is -6.021 points (95% CI $[-17.780, 5.738]$, exact sign-flip $p = 0.1250$, Holm $p = 0.2500$). Averaging over families, the multi-agent main effect is -0.521 points (CI $[-1.441, 0.399]$, exact $p = 0.1875$, Holm $p = 0.2500$). Neither main effect is distinguishable from zero. The difference-in-differences interaction is -1.792 points: role separation moves from 0.375 points worse without the market to 1.417 points better with it. Its exact sign-flip test is $p = 0.0156$ and survives the three-test Holm family at $p = 0.0469$, but its paired- t interval $[-3.836, 0.252]$ crosses zero. Seven windows have a negative interaction and five are exact zeros, with none positive; this directional consistency explains the exact result, while unequal magnitudes widen the parametric interval. We therefore call the interaction a *qualified positive* for market complementarity, not evidence that multi-agent reasoning wins overall or a precise estimate of its magnitude.

Execution was clean: zero LLM or runtime errors and zero invalid Single rulings. Multi emitted four invalid rulings, all absorbed by the same-family deterministic floor fixed on validation. Across its 24 test runs Multi used 3,663 calls and 10.93 wall-hours, against 1,239 calls and 1.62 wall-hours for Single. The registered amendment reduced three inference seeds to seed 17 to finish the experiment; decoding noise is consequently inseparable from between-window variation.

Validation selects ageing fair-share with queue-proportional sizing at 10.49% violations for the ordering family, and the deadline-priced auction with queue-proportional sizing at 9.87% for the auction family, each by validation violations alone with a deterministic tie-break, and each carried unchanged to the reserved split. The registered inferential family contains only the two main effects and their interaction; we do not add an unregistered selector-against-fixed hypothesis after seeing the test outcomes. Section IV-D instead reports the deterministic action space descriptively.

D. Inside the action space: sizing and pricing dominate ordering

The 32 deterministic controls are complete on both splits: every configuration ran on all 12 validation and all 12 test windows. They characterise the action space the selectors choose within, independently of any selector. All figures below are paired over the 12 held-out windows.

Two facts constrain what any selector could contribute. First, the ordering rule is the least consequential part of the action: the four orderings span 14.48 to 15.44 points, under one point, while sizing under a fixed ordering spans 15.07 to 47.06, a factor of three. Ordering is not literally inert, but it is worth roughly a thirtieth of what sizing is worth, which reproduces §IV-B inside the larger 8×4 space, on held-out windows, and with deadline violations rather than slowdown as the outcome. Second, within the auction family the clearing mechanism is held fixed and only what a claim is *priced on* moves the outcome: pricing deadline pressure gives 8.11 points against 14.48–15.03 for wait, priority, and fair-share pricing. The decisions that matter are therefore how much capacity each job receives and what its claim is priced on, not the order in which the queue is scanned.

The difference between the two families is a tail effect and has to be reported as one. The best auction configuration beats the best ordering configuration by 6.38 points on average, while the median difference across the 12 windows is exactly zero: seven windows tie to the digit, four move by 0.5 to 3.8 points, and one moves by 69.4, from 91.0% violations to 21.6%. That window is the only one in the set where the cluster is overwhelmed, and it is where almost the entire mean difference lives. A mean-only reading would present a one-window rescue as a general property of auction clearing.

One note on how this table reached completeness, since it bears on the value of running the checks at all. Six configurations on one test window initially aborted, the capacity assertion reporting 81 devices allocated from a pool of 80. The cause was in the checker, not the scheduler: the simulator retains a killed job’s assignment in its mirror for the job’s callback while already offering those devices as free, and the assertion counted the stale record. Completed jobs were already excluded and killed ones now are. Re-running the six configurations after that correction produced no violation, and no allocation in this paper ever exceeded the pool. The episode is worth a sentence because a capacity assertion that fires on a bookkeeping artifact is indistinguishable, from the

outside, from one that catches a real over-allocation, and only reproducing it separates the two.

E. Counterfactual oracle and LoRA protocol

To test whether supervision rather than architecture is the bottleneck, we clone sampled simulator states and execute all 16 policy-sizing actions in the relevant family from the same state, with the same future arrivals and seed. The lowest deadline-violation action supplies an offline label; future outcomes are never included in the online prompt. We sample two states per window and family. Four states from one training window were discarded under the superseded capacity assertion described in §IV-D; they are recoverable and their inclusion would slightly change the bound below. The resulting 1,092 auditable rollouts form 44 complete training examples and 24 validation examples, with disjoint windows and no oracle-only fields in model input.

Enumerating from sampled states, the best action per state improves on the fixed policy by 2.31 points on training and 2.03 on validation for the ordering family, and by 0.51 and 0.07 for the auction family. The median gain is zero in every one of those four cells: selection matters in a minority of states, especially in the non-market family, while a strong fixed market policy leaves almost no mean headroom. We fine-tune Qwen3-8B with rank-16 LoRA on assistant spans only and judge it on the 24 held-out states by valid-action rate, oracle-action agreement, mean counterfactual regret, and improvement over the best fixed training action. This evaluation is deliberately offline: it determines whether the model learned the selector before another expensive closed-loop sweep is warranted.

It did not. On the 24 held-out states, the deterministic policy carries 1.05 points of mean regret against the counterfactual best action, a zero median, 75.0% of states within 0.1 points of that action, and no invalid actions. Zero-shot, the model carries 8.70 points, a 0.8 median, 45.8% near-optimal, and 11 of 24 illegal or unparseable actions. Fine-tuned, it carries 8.78 points, the same median, 41.7% near-optimal, and the same 11 failures.

Neither language-model arm is competitive. Both carry roughly eight times the deterministic policy’s regret, both leave a median state 0.8 points short of the best available action where the deterministic policy leaves it exactly at the optimum, and both emit an unparseable or illegal action on 11 of 24 states, which the deterministic policy never does. Fine-tuning moves nothing: mean regret is marginally worse than zero-shot, the median is identical, near-optimal states fall, and the invalid count is unchanged. An initial reasoning-enabled decode was worse still, at 23 of 24 invalid, so both reported arms use the non-thinking template for this strict-JSON task; invalid outputs are conservatively scored as the worst candidate action.

The closed-loop numbers agree with the offline screening. On the validation split the single referee reaches 11.27% deadline violations on the ordering family and 10.70% on the auction family, and the role-separated referee 11.93%

on the ordering family, against 10.49% and 9.87% for the deterministic policies selected on that same split. Those fixed policies were chosen on these windows, so this is not a held-out comparison and we do not test it; the held-out evidence is the held-out regret above, and it points the same way. Taken together, a small language model asked to choose scheduling policies from observable state — zero-shot or fine-tuned, alone or split into demand and supply roles — does not match a single well-chosen deterministic policy on this workload, and a closed-loop fine-tuned sweep is not warranted by this screening.

This is a statement about the numeric rationing decision, not about language models in schedulers. The same deterministic policy that dominates here is helpless when the deciding fact is not in the numbers at all, which is the case we turn to next.

F. Post-core debate repair and outcome-gated rotation

The first bounded protocol produced a deliberately mixed four-training-window check: relative to the single selector it changed deadline violations by 0.0, −9.8, −14.2, and +0.5 points as offered load increased. Stable selection therefore did not imply a win, and no average or held-out claim was made. This motivated a separately versioned protocol that lets language models propose changes but makes observed outcomes authoritative. Every 1,800 simulated seconds, Demand and Supply independently propose a role-valid auction ordering or abstain; their two calls run in parallel. A Referee may only hold the saved incumbent or authorize one requested challenger—it cannot directly name or switch to another ordering. Sizing is outside all three schemas and is reviewed by a deterministic hysteresis controller every 300 simulated seconds. Thus an ordering epoch still costs three calls, not an additional reflection round.

An authorized challenger receives a 30-minute trial and, if its role-specific queue, deadline-pressure, tail-wait, and concentration checks pass, a second 30-minute probation. Failure restores the incumbent; only two passes accept the challenger. Protocol v2.4.3 corrects the learning record to use deltas from the phase that actually passed or failed, labels rollback as negative evidence about the *challenger*, and removes an exact incumbent-to-challenger transition from the Referee’s allowed actions for four hours after two consecutive rollbacks. Rotation never chooses a branch randomly. The following training-window checks are exploratory: the rule was revised after inspecting earlier training runs, and the 24-window sweep was explicitly run despite a frozen two-window gate to meet a presentation deadline.

a) Two-window v2.4.3 repair check.: The preregistered training-window pilot is running. This sentence is generated and will be replaced with measured results immediately after both rows complete; no pending number is used elsewhere in the paper.

b) Twenty-four-window v2.4.3 sweep.: The explicitly authorized training-window sweep is running. This generated placeholder will be replaced by the analyzed paired results before they are presented as evidence.

V. FUTURE WORK AND THREATS TO VALIDITY

Small, single-opened reserved split. Twelve paired windows at one inference seed support a directional interaction, not a precise effect: five windows tie, the paired- t interval crosses zero, and the seed amendment merges decoding noise into window differences. More windows, seeds, and per-state counterfactual regret are needed.

Synthetic elastic outcomes. Amdahl speed-up is anchored to each observed runtime and swept, but reconfiguration cost and intra-node effects are absent. Deadlines are reproducible synthetic targets from requested walltime, while slowdown uses unknown true runtime; we therefore also report waiting time and device-hours.

Narrow model evidence. The offline comparison has 24 held-out states and one 8B fine-tuning run while the closed loop uses 14B. It screens this recipe; it does not prove supervision cannot work.

Adaptive reuse of training windows. The rotation protocol and its backoff were revised after examining failures on the same training windows used for the exploratory sweep. Those results measure repair and mechanism behavior, not generalization; a frozen validation/test evaluation is required before making a confirmatory performance claim.

VI. CONCLUSION

Once jobs can be resized, occupancy stops being a safe objective: sublinear speed-up buys busy-ness rather than completed work. Device-hours per completed work belongs in the headline.

Our title question therefore has a qualified answer. An 8B selector carries 8.70–8.78 held-out regret against 1.05 for deterministic code, and role separation has a null main effect on the reserved split despite a market interaction. The exploratory bounded-debate sweep gives two positive stress windows (−9.8, −14.2 deadline points) and one negative (+0.5): a basis for confirmatory testing under contention, not a general multi-LLM advantage.

Where a language model might still earn its place is the case this workload cannot express: an exception whose content is text, such as an operator instruction or a declaration that a note contradicts. The accounting log carries 29 fields and none is free text, and no public GPU trace we surveyed carries operator-authored rationale, so that case cannot be measured here at all. We flag it as the open direction. The recommendation follows the evidence rather than the architecture: keep a validated deterministic default, count every malformed answer and fallback, and admit a language model only where a held-out regret test and a paired closed-loop comparison beat that default after inference cost.

REFERENCES

- [1] G. Ding, S. Yang, H. Lin, Z. Chen, and J. S. Yang, “LLM-driven adaptive cloud resource scheduling: bridging reasoning intelligence with optimization guarantees,” *IEEE Open J. Comput. Soc.*, 2026.
- [2] S. Samsi et al., “The MIT Supercloud dataset,” *IEEE High Performance Extreme Computing Conf. (HPEC)*, 2021, pp. 1–8.
- [3] H. Mao et al., “Resource management with deep reinforcement learning,” *HotNets*, 2016.
- [4] H. Mao et al., “Learning scheduling algorithms for data processing clusters,” *SIGCOMM*, 2019.
- [5] A. Tarraf et al., “Malleability in modern HPC systems: current experiences, challenges, and future opportunities,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 9, pp. 1551–1564, 2024.
- [6] Q. Weng et al., “MLaaS in the wild: workload analysis and scheduling in large-scale heterogeneous GPU clusters,” *NSDI*, 2022 (cluster-trace-gpu-v2020).
- [7] T. Özden, T. Beringer, A. Mazaheri, H. M. Fard, and F. Wolf, “ElastiSim: a batch-system simulator for malleable workloads,” *ICPP*, 2022, pp. 1–11.
- [8] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, “Improving factuality and reasoning in language models through multiagent debate,” *ICML*, 2024, pp. 11733–11763.
- [9] X. Wang et al., “Self-consistency improves chain of thought reasoning in language models,” *ICLR*, 2023.
- [10] L. Zheng et al., “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” *NeurIPS Datasets and Benchmarks Track*, 2023.
- [11] B. An, “Automated negotiation for complex multi-agent resource allocation,” Ph.D. dissertation, Univ. of Massachusetts Amherst, 2011.
- [12] M. Croitoru, C. Croitoru, and G. Ganesh, “Prediction-based selective negotiation for refining multi-agent resource allocation,” *ICAART*, vol. 1, 2025, pp. 656–662.